# SW Engineering CSC 648 Fall 2025

## Tutor Connect

## Team 02

## Milestone 1: High-Level Specs and Use Cases

**Team Lead:** Ranjiv Jithendran | rjithendran@sfsu.edu

**Front-End Lead:** Adea Mulaku

**Back-End Lead:** Bao Than

**GitHub Master:** Dhvanil Bhagat

**Scrum Master:** Harry Wong

Submitted:

Revised:

# 1. Executive Summary

**Problem & motivation:**
SFSU students often struggle to find trustworthy, course-specific academic help quickly. Generic tutor marketplaces don't reflect SFSU's curriculum, make it hard to verify that tutors are actually SFSU students, and bury students in irrelevant profiles. This wastes time, especially near midterms and finals, when targeted help matters most.

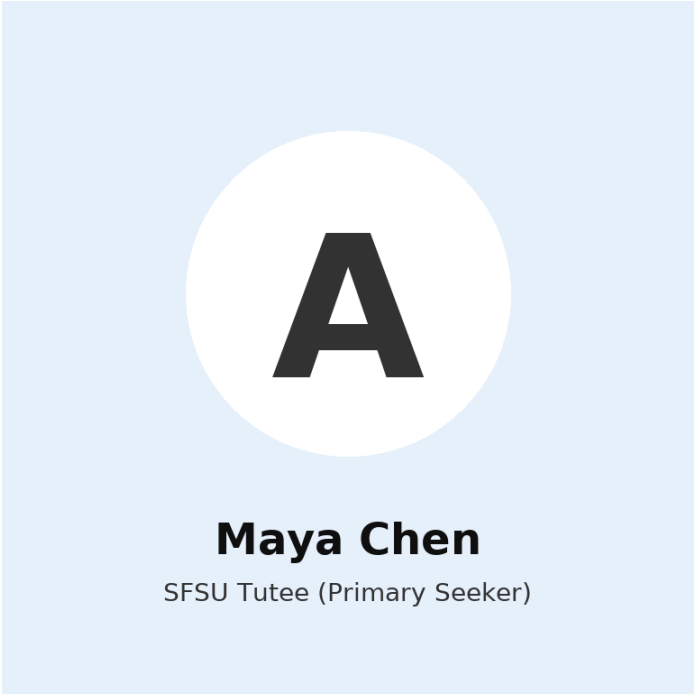
**Our solution:**
TutorConnect is a desktop-first web application for the SFSU community. Students can **search by SFSU course code** (e.g., "CSC 340"), browse **approved** tutor listings, view profiles with clear availability, and initiate contact through a **single, in-site message**, so no email or chat is required. Student tutors create or update listings that remain **pending** until an **admin** approves them for quality and policy compliance. No payment features are implemented or simulated in the UI.

**SFSU-specific value:**
The platform is **SFSU-only** for posting and messaging (accounts must use a valid @sfsu.edu). Search pivots around **course codes** and subjects students actually take at SFSU, helping them match quickly and confidently. Admin approval plus a simple flagging mechanism improves safety and quality.

**Team:**
Team 02 is a cross-functional group of SFSU student software engineers (front end, back end, and DevOps). We're building a maintainable, standards-based system (React (Vite) + Node + MySQL + Python) that easily evolves in later milestones with instructor feedback.

## 2. Personae

**Persona A — "Maya Chen," SFSU Tutee (Primary Seeker)**




**Bio:** 18–19, first-year Biology major living in dorms. Studies most evenings after lab, splits time between the J. Paul Leonard Library desktops and her phone while moving across campus.
**Goals:** Find a tutor by course code (e.g., BIOL 230, MATH 227), match availability around labs, and message without leaving the site. Wants quick proof of legitimacy (SFSU email, past course performance).
**Behaviors & skills:** Comfortable with keyword search, filters, and skim-reading listings on mobile between classes; saves candidates to review on a desktop later.
**Pain points:** Generic marketplaces don't map to SFSU course numbers, so she wastes time decoding "equivalents." Calendars often don't line up, causing back-and-forth DMs. Worried about ghosting and no-shows.
**Accessibility & constraints:** Budget-sensitive; prefers Zoom or library study rooms. Needs calendar sync with her SFSU class schedule.
**What "good" looks like:** Finds 2–3 tutors tied to BIOL 230 within 5 minutes, sees live overlap in availability, messages once, and books the first session.
**Quote:** "Just show me someone who's done well in my exact course number and has time when I'm free."

**Persona B — “Diego Alvarez,” SFSU Student Tutor**


**Bio:** 21–22, senior CS major who recently aced CSC 340 and CSC 413; wants flexible income that fits club meetings and capstone.
**Goals:** Publish a clean listing with course codes, short bio, and optional media (whiteboard clip, PDF cheatsheet); control a weekly availability grid; get inquiries in one dashboard.
**Behaviors & skills:** Comfortable editing listings, uploading files, and recording short Loom-style clips. Uses Google Calendar religiously.
**Pain points.** Approval queues feel opaque and slow; reschedules kill his utilization; generic sites bury SFSU-specific creds under vague tags. Wants fewer “Are you available at…?” DMs.
**Incentives.** Keeps rates competitive if he can reliably fill 6–8 hours/week. Doesn’t want to wrangle payments; prefers platform-tracked requests and receipts (even if payment happens off-platform per class rules).
**What “good” looks like**. Auto-approved updates (or clear SLAs), instant conflict detection, and a unified inbox with templates.
**Quote.** “If students can see I crushed CSC 340 at SFSU and when I’m free, they’ll book—no endless DMs.”

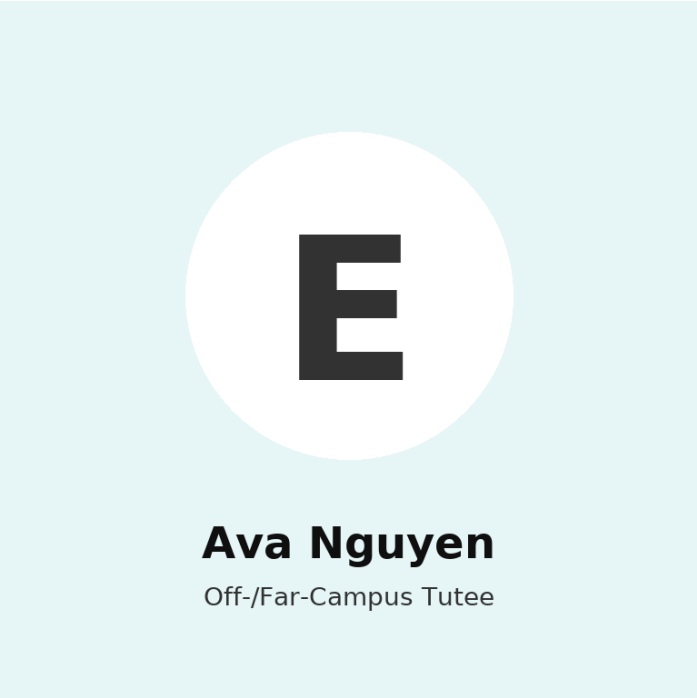
**Persona C — “Samira Patel,” Admin / Moderator**


**Bio.** TA/staff designee volunteering a few hours a week. Works in short bursts between responsibilities.
**Goals.** Keep the catalog clean and safe: approve/reject pending listings fast, triage flags, and maintain an audit trail for reviews/edits.
**Behaviors & skills.** Comfortable with queues, filters (by course, date, status), and bulk actions. Prefers concise signals (SFSU email verified, course transcript snippet uploaded).
**Pain points.** Duplicate/low-quality submissions, unclear media rights, and missing action history; wants reversible actions and breadcrumbs for compliance.
**Operational constraints.** Limited time; needs keyboard-first moderation and canned responses.
**What “good” looks like.** 10–15 approvals in one sitting with confidence, zero policy violations, and a clear log that explains why each decision was made.
**Quote.** “Give me strong signals and bulk tools so I can keep the catalog trustworthy in 15 minutes.”

**Persona D — “Jordan Lee,” SFSU Tutee (Learning Disability)**


**Bio.** 19–20, second-year Business major with diagnosed ADHD and reading-processing accommodations via DPRC. Often studies in 45-minute blocks.
**Goals.** Find tutors for specific SFSU courses who can adapt pacing, provide structured outlines, and respect accommodation needs (frequent breaks, shared notes).
**Behaviors & skills.** Uses mobile to shortlist; relies on visual indicators (badges, icons) and plain-language summaries. Favors recorded mini-recaps after sessions.
**Pain points.** Listings rarely state neurodiversity experience; text-heavy pages are overwhelming; schedulers don’t reflect shorter session blocks.
**Accessibility & constraints.** Needs alt text, clean headings, reduced cognitive load, and the option to book 30–45 minute sessions.
**What “good” looks like.** Filters by “ADHD-friendly / DPRC-aware,” sees tutors’ adaptations (timed breaks, structured agendas), and books quickly with confidence.
**Quote.** “Tell me who actually *knows how* to teach students like me—and let me book shorter, focused sessions.”

**Persona E — “Ava Nguyen,” SFSU Tutee (Off- or Far-from-Campus)**




**Bio.** 22, commuter living in Daly City; heavy job schedule and long transit times.
**Goals.** Find tutors by SFSU course code who can meet off-peak or online; prefers options near BART, Zoom, or hybrid plans (first session in person, rest remote).
**Behaviors & skills.** Skims on mobile during commutes; uses map filters and travel-time hints (e.g., “<15 min from Daly City BART”).
**Pain points.** Hard to know who can commute or offer remote; availability often assumes on-campus afternoons; messaging chains to negotiate location are tedious.
**Constraints.** Tight windows (early mornings, late evenings); needs reliable reschedule flow without penalties.
**What “good” looks like.** Toggles “Remote / Near BART,” sees travel-friendly tutors with matching windows, and books instantly with location details baked in.
**Quote.** “If I can’t make it to campus, I still need an SFSU-specific tutor who fits my commute life.”

---

# 3. High-Level Use Cases

**UC-1 — Search by SFSU Course Code (Tutee; SFSU-specific)**

**Role:** Tutee

A student needs help in a specific course (e.g., “CSC 340”) and searches by course code, optionally adding filters like modality and evening availability. The system returns only approved listings that explicitly include that code, helping the student quickly identify relevant tutors.

**UC-2 — Browse & Filter Tutors (Tutee)**

**Role:** Tutee

The student casually browses tutor listings and applies filters (subject, modality, availability window) and sorting (relevance/recency). The system shows a paginated list of approved listings that match.

**UC-3 — View Tutor Profile & Availability (Tutee)**

**Role:** Tutee

From search/browse, the student opens a tutor profile to review bio, subjects, course codes, availability windows, and media. The student compares a couple of candidates and selects one to contact.

**UC-4 — Send One In-Site Message to a Tutor (Tutee → Tutor)**

**Roles:** Tutee → Tutor

After selecting a tutor, the student sends a single, in-site message (course/topic + preferred time window). The system stores the message and surfaces it on both dashboards; no external email or chat is used.

**UC-5 — Save Tutors & Use the Tutee Dashboard (Tutee)**

**Role:** Tutee

While exploring, the student saves/unsaves tutors to a shortlist and later checks the Tutee Dashboard to see saved items and the status of their sent message.

**UC-6 — Leave a Simple Rating/Review (Tutee; optional feature)**

**Role:** Tutee

After receiving help (arranged outside the app via the one-round contact), the student returns to the site and leaves a simple rating/review for the tutor. The system associates the rating with the listing (pending moderation if needed) to aid future search/browse decisions.

**UC-7 — Create/Update Tutor Listing → Pending Review (Tutor)**

**Role:** Tutor

A student tutor signs in with an @sfsu.edu email and creates/edits a listing (bio, subjects, course codes, availability, media). On submit, the listing status becomes Pending for admin review; edits to approved listings also revert them to Pending.

**UC-8 — Monitor Listing & Inquiries in Tutor Dashboard (Tutor)**

**Role:** Tutor

The tutor opens the Tutor Dashboard to see listing status (Draft/Pending/Approved/Rejected), views, and any incoming tutee message. The tutor updates availability or profile content as needed to improve visibility.

**UC-9 — Review & Moderate Content (Admin)**

**Role:** Admin/Moderator

The admin signs in and opens a moderation queue filtered by Pending status (and other facets like subject or age). For each listing, the admin checks text/media for policy compliance and approves or rejects with an optional note; they can also hide/remove inappropriate items reported by users.

**UC-10 — Manage Users & Audit Actions (Admin)**

**Role:** Admin/Moderator

When repeated violations occur, the admin suspends a user account and/or deletes offending content. The system records an audit log (who/what/when) and updates any open flags to Resolved with the action taken.

---

# 4. Main Data Items & Entities

**User (Abstract)**

Meaning: Any authenticated person using the system. Specialized into Student, Tutor, and Admin.

Key attributes: userId, name, email, role, status (active/suspended), createdAt.

Usage: Authorization, ownership of actions (bookings, messages).

Relationships: One-to-one with Profile; one-to-many with Booking, Message, Review.

Visibility: Self + Admin; limited fields visible to counterparty tutors/students via Profile.

**Student**

Meaning: A User seeking help; inherits User.

Key attributes: studentId (alias of userId), major, graduationTerm.

Usage: Booking sessions, course linking, reviews.

Relationships: Has many Bookings, Courses (via Enrollment).

Visibility: Admin; tutors can view essentials for booked sessions.

**Tutor**

Meaning: A User offering tutoring; inherits User.

Key attributes: tutorId (alias of userId), verificationStatus (approved/pending), hourlyRate, pictureID

Usage: Availability publishing, receiving bookings, and payouts.

Relationships: Has Availability, Subjects, Reviews, Bookings.

Visibility: Public profile details; admin sees verification info.

**Admin**

Meaning: Staff user managing platform policy and disputes; inherits User.

Key attributes: adminId (alias of userId), permissions (moderation, refunds).

Usage: Oversight, audit, configuration.

Visibility: Full.

**Subject**

Meaning: Topic areas a tutor can teach or a student can request.

Key attributes: subjectId, name (e.g., “Data Structures”), category (STEM, Writing), description.

Usage: Search/filter; mapping to Courses.

Relationships: Many-to-many with Tutor; optional mapping to Course.

Visibility: Public.

**Course (SFSU-Specific)**

Meaning: Official SFSU course a session may reference.

Key attributes: courseId, department (e.g., CSC), courseNumber (e.g., 648), section, classNumber (SFSU “Class Number”), term (e.g., Fall 2025), instructorName.

Usage: Enables SFSU-custom flows (e.g., match by Class Number), course-aware search, reporting.

Relationships: Many-to-many with Student via Enrollment; optional link to Subject; used by Session for context.

Visibility: Public catalog info; enrollment details private.

**Notification**

Meaning: System alerts (booking updates, reminders).

Key attributes: notificationId, recipientId, channel (email/push/in-app), title, body, sentAt, status.

Usage: Keeps users informed and reduces no-shows.

Relationships: Belongs to User; references Booking when applicable.

Visibility: Recipient + Admin (logs).

**Report/Flag**

Meaning: Community moderation signal about a user, message, or review.

Key attributes: reportId, reporterId, targetType (user/message/review), targetId, reason, actionTaken.

Usage: Safety and policy enforcement.

Relationships: Points to the target entity; reviewed by Admin.

Visibility: Admin; aggregated impact may affect public visibility

**Bookmarks**

Meaning:

A private “save for later” marker a user creates on an entity they want to revisit (e.g., a Tutor, Subject, Course, or specific Booking thread).

Primary use cases:

Students shortlist tutors before booking.

Students/tutors save Message threads or Attachments for quick access.

Students save Subjects/Courses they routinely need help with.

Key attributes: bookmarkId, userId (owner), targetType (e.g., Tutor, Subject, Course, MessageThread, Booking), targetId, createdAt, notes, labels,

Relationships:

Belongs to User (owner).

Optionally belongs to BookmarkFolder (user-defined grouping).

Points to a single target entity via (targetType, targetId) polymorphic reference.

Notification system only for local UI cues (no external alerts).

Visibility & Permissions:

Always private to the owner; not shown to other users.

**Availability:**

Meaning:

Time windows a Tutor offers for booking (recurring or one-off), including mode and days.

Key attributes:availabilityId, tutorId, timezone, modeOfInstruction (online, inPerson, hybrid)

daysOfWeek, slotLengthMinutes, bufferBeforeMinutes, bufferAfterMinutes, effectiveStartDate, effectiveEndDate, repeatRule (optional), exceptions (optional), locationId (in-person), meetingLinkTemplate (online/hybrid)

capacity (default 1), leadTimeMinHours, cancelCutoffHours, subjectIds (optional), courseIds (optional), status, createdAt, updatedAt,

Relationships:

Belongs to Tutor; used by Booking to generate valid slots.

Visibility & perms:

Public read-only in search; editable by Tutor/Admin.

Meeting link/location shown after booking confirmation.

**Reviews:**

Meaning:

Student feedback on a completed Session/Tutor (rating + comment) used for quality and search ranking.

Key attributes:reviewId, sessionId, tutorId, studentId, rating (1–5), title (optional), comment

createdAt, updatedAt, visibilityStatus (public, hidden, flagged), helpfulCount (derived), reportedCount (derived), response (optional tutor reply**)**

Relationships: Belongs to Session; references Tutor and Student. Can be reported via Report/Flag entity; aggregates to Profile rating summary.

**Student Profile:**

Meaning: Basic information about a student (registered user but not a tutor), like name, major, and picture

Key attributes: studentID, majorID, pictureID

Relationship: Belongs to registered users that are not tutors, allows tutors to see which user has booked them, and provides some basic information about them.

**Tutoring Sessions:**

Meaning: stores information on which tutor is booked when, and for users, shows which sessions they have

**Course Teacher:**

Meaning: Stores which professor the tutor took a specific course with

# 5. High-Level Functional Requirements

## 5.1 Unregistered Users (Public)

1. **Public browse**
   The system will allow any visitor to browse a paginated list of approved tutor listings.
2. **Public profile view**
   The system will display a public tutor profile with bio, subjects, course codes, availability snapshot, and approved media.
3. **Keyword/subject search**
   The system will support searching listings by keyword and by subject/category.
4. **SFSU-specific course-code search**
   The system will support searching listings by SFSU course code (e.g., “CSC 340”).
5. **Filter and sort**
   The system will provide filters (e.g., course/subject, modality, availability window) and sorting (e.g., relevance, most recent) for search/browse results.

## 5.2 Registered Users (SFSU email required)

1. Inherits all functions of unregistered users*
2. **SFSU registration**
   The system will allow user registration only with an email ending in **@sfsu.edu**.
3. **Sign-in / sign-out / session**
   The system will allow registered users to sign in/out and maintain an authenticated session to access protected features.
4. **Profile management**
   The system will allow a signed-in user to view and edit their profile and to opt in to the **Tutor** role.
5. **One-round in-site message (tutee → tutor)**
   The system will allow a signed-in tutee to send one in-site message to a selected tutor; subsequent attempts to the same tutor will be blocked.
6. **Tutee dashboard**
   The system will present a tutee dashboard showing sent message(s) with status/read, saved tutors, and recently viewed listings.
7. **Save/unsave tutor**
   The system will allow a signed-in user to save and unsave tutor listings to a personal shortlist.

8. **Create/edit tutor listing**
   The system will allow a signed-in tutor to create and edit a listing including bio, subjects, course codes, availability, modality, and media.
9. **Listing status workflow**
   Upon create/edit, the system will set the listing status to Pending and keep it hidden from public search until **Approved** by an admin.
10. **Tutor dashboard**
    The system will present a tutor dashboard showing listing status (Draft/Pending/Approved/Rejected), views/interest, and received inquiries.
11. **Availability management**
    The system will allow tutors to manage recurring availability windows that are surfaced in profile and search filters.
12. **Report/flag content**
    The system will allow any signed-in user to file a flag against a listing or media item with a short reason.
13. **Simple rating/review**
    The system may allow a tutee to submit a simple rating/review for a tutor after receiving help; submitted reviews will be subject to moderation before public display.

## 5.3 Administrators / Moderators

1. **Approve/reject listings**
   The system will allow admins to review Pending listings (text + media) and approve or reject them with an optional note to the tutor.
2. **Moderate content and users**
   The system will allow admins to hide/remove listings or media, and to suspend/reinstate user accounts when policy violations occur.
3. **Manage and resolve flags**
   The system will provide admins with a queue of user flags and allow them to mark each flag **Resolved** with the action taken.
4. **Admin search & audit trail**
   The system will allow admins to search listings/users and view an audit trail (who/what/when) of moderation actions.

**Notes***

- 4, 6, 9, 12 and 13 explicitly carry the SFSU-specific and one-round in-site message constraints forward from the specification.

## 6. Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. All or selected application functions shall render well on mobile devices (no native app to be developed)
4. Posting of tutor information and messaging to tutors shall be limited only to SFSU students
5. Critical data shall be stored in the database on the team's deployment server.
6. No more than 50 concurrent users shall be accessing the application at any time
7. Privacy of users shall be protected
8. The language used shall be English (no localization needed)
9. Application shall be very easy to use and intuitive
10. Application shall follow established architecture patterns
11. Application code and its repository shall be easy to inspect and maintain
12. Google Analytics shall be used
13. No e-mail clients shall be allowed. Interested users (clients) can only message to service providers via in-site messaging. One round of messaging (from client to service provider) is enough for this application. No chat functions shall be developed or integrated
14. Pay functionality (e.g., paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items
16. Media formats shall be standard as used in the market today
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only" at the top of the WWW page Nav bar. (Important so as to not confuse this with a real application).

## 7. Competitive Analysis

| Product/Platform | SFSU course-code search | Tutor verification / reviews | Availability | On-platform messaging | Modality (in-person / online) | Notes / caveats |
|---|---|---|---|---|---|---|
| Our App (SFSU Tutor Connect) | ✓ (maps to SFSU codes) | ✓ (SFSU email + course proof) | ✓ (live availability + conflict warnings) | ✓ (unified inbox) | Both (in-person + Zoom) | Differentiators: SFSU code mapping, DPRC-friendly filters, BART/commute filters |
| Wyzant | ✗ (subject-level) | ✓ (marketplace reviews) | ✓ (booking) | ✓ (chat) | Both | Large marketplace; no campus-specific mapping |
| Superprof | ✗ (subject-level) | ✓ (ratings) | ≈ (arrange off-thread) | Contact gated by Student Pass | Both (varies by tutor) | Messaging/contact behind monthly paywall |
| TutorOcean | ✗ (subject-level) | ✓ (vetted marketplace) | ✓ (booking + notifications) | ✓ (platform comms) | Both (online classroom + in-person) | Campus-agnostic; strong online classroom |
| SFSU Tutoring & Academic Support Center (TASC/LAC) | ✓ (campus courses) | ✓ (campus-hired; no public reviews) | ✓ (appointments via Navigator) | ✗ (not a marketplace) | Both (LIB 220 + Zoom) | Institutional service; set hours; limited catalog |
| Student-run Discord/Reddit groups (indirect) | ✗ (informal) | ✗ | ✗ (ad-hoc) | ✓ (DMs/threads) | Usually, online / meetups vary | High variability; no safeguards |

## Brief Analysis:

Marketplaces (Wyzant, TutorOcean) cover breadth, reviews, and basic booking/chat but are campus-agnostic—no exact SFSU course-code mapping. Superprof introduces a contact paywall (Student Pass), adding friction for price-sensitive students. SFSU TASC/LAC provides verified, free support with structured appointments but is not a marketplace (no public reviews or flexible matching). Your wedge: SFSU-specific course-code mapping, live availability with conflict warnings, DPRC-friendly filters, and commute-aware options (e.g., near BART, Zoom).

---

# 8. High-level System Architecture

## Logical architecture:

**Client (Web UI)** — Desktop-first, responsive SPA built with Vite. Loads GA4 and the required banner text on every page.

**Edge / Reverse Proxy** — Nginx terminates TLS, serves static front-end assets, and reverse-proxies /api/* to the Python app.

**Application Server (API)** — Python service exposing REST endpoints for auth, listings, messaging, moderation, and dashboards. Enforces SFSU email rule and one-round tutee→tutor messaging.

**Database** — MySQL 8 persists all critical data (users, tutor profiles, listings, approvals, messages, flags, audit logs).

**Static media** — Uploaded images/video referenced by listings. (Phase 1: stored on the EC2 instance; Phase 2+: move to S3 and serve via Nginx.)

**Observability** — App + access logs (Nginx + API), health check endpoint, basic metrics; GA4 for usage analytics.

**Request flow:** Browser ⇄ Nginx (TLS, static, cache) ⇄ Python API ⇄ MySQL (+ filesystem/S3 for media)

## Technologies:

**Frontend:**

Vite, Tailwind CC, SPA, React 18 (recommended), React Router 6, Fetch/Axios, Google Analytics 4 via gtag.js (global)

**Backend:**

Python 3.12

FastAPI 0.11x

ASGI server: Uvicorn (dev) / Gunicorn + Uvicorn workers (prod)

Validation & schemas: Pydantic v2

ORM: SQLAlchemy 2.x (with Alembic migrations)

Auth: Signed HttpOnly cookies (session/JWT), role middleware

Security: python-jose (if JWT), passlib for hashes, rate-limit middleware

File uploads: presigned flow (Phase 2) or validated direct upload (Phase 1)

**Database & Storage:**

MySQL 8.0 (InnoDB, utf8mb4)

Backup strategy: nightly logical backups + weekly restore test

Media: EC2 filesystem (Phase 1) → S3 bucket (Phase 2+), metadata in DB

**Infrastructure / DevOps:**

AWS EC2 (Ubuntu 24.04 LTS) single instance to start

Nginx 1.26 (TLS via Certbot)

Process manager: systemd or supervisor (Gunicorn service)

CI/CD: GitHub Actions

Environment secrets via .env on server (no secrets in repo)

Browsers supported (per NFR)

Latest two versions of Chrome and Firefox (desktop)

Mobile layouts verified on iOS Safari & Android Chrome (responsive only)

**Security & compliance**

1. SFSU-only posting/messaging: enforced at signup (email.endswith('@sfsu.edu')) and on protected endpoints.
2. One-round in-site messaging: service rule blocks additional tutee→tutor messages after the first.
3. No email / no chat / no payments: email clients and payment UI are not implemented or simulated.
4. Critical data persisted in DB: users, listings, approvals, messages, flags, audit logs in MySQL.
5. Basic hardening: HTTPS everywhere, HSTS, Helmet-style headers via Nginx, strict CORS, rate limits on auth/messaging, parameterized queries via ORM, least-priv DB user, regular backups.

---

# 9. Use of GenAI Tools

1. Personae Creation
    - ChatGPT 5 Thinking
    - Used ChatGPT to generate fictional persona based on the requirements outlined in section 2 of the M1 instruction document (HIGH usefulness)
    - Prompt: (uploaded the instruction document) “Analyze this document and then create 3-5 personas for a tutoring application that connects tutors and students on SFSU campus, use SFSU related data points and make sure to include what the section for personae is asking”

---

## 10. Team Members and Roles

**Team Lead:** Ranjiv Jithendran

**Front-End Lead:** Adea Mulaku

**Back-End Lead:** Bao Than

**GitHub Master:** Dhvanil Bhagat

**Scrum Master:** Harry Wong

## 11. Team Lead Checklist

1. So far all team members are fully engaged and attending team sessions when required: **ON TRACK**
2. Team found a time slot to meet outside of the class: **DONE/OK**
3. Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing: **ON TRACK**
4. Team reviewed class slides on requirements and use cases before drafting Milestone 1: **DONE/OK**
5. Team reviewed non-functional requirements from “How to start…” document and developed Milestone 1 consistently: **DONE/OK**
6. Team lead checked Milestone 1 document for quality, completeness, formatting and compliance with instructions before the submission: **DONE/OK**
7. Team lead ensured that all team members read the final M1 and agree/understand it before submission: **DONE/OK**
8. Team shared and discussed experience with GenAI tools among themselves: **DONE/OK**
9. Github organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.): **DONE/OK**